

ESP-32 y RS-485: alternativa de bajo costo para sistemas de comunicación maestro-esclavo

Emiliano Villarreal, Sabrina Higgimbotton, Roberto M. Murdocca, Sergio F. Hernandez Velazquez
Departamento de Electrónica

Universidad Nacional de San Luis, San Luis, Argentina

{emiliano02villarreal, sabrinahiggimbotton, mmurdocc, sergio.sfhv}@gmail.com

Resumen—Este proyecto presenta el diseño e implementación de una red de comunicación RS-485 con arquitectura maestro-esclavo, orientada a su uso en entornos educativos. El sistema está conformado por un nodo maestro ESP32 y tres nodos esclavos, también basados en ESP32. Cada nodo esclavo integra sensores y actuadores como un DHT11, un sensor ultrasónico HC-SR04, un buzzer, un motor SG90 y LEDs. La comunicación se realiza mediante tramas de 8 bits con estructura definida, utilizando una interfaz Half-Duplex.

Index Terms—RS-485, ESP32, comunicación serial, maestro-esclavo, sistemas embebidos.

I. INTRODUCCIÓN

RS-485 es un estándar ampliamente utilizado en la industria por su robustez, bajo costo y capacidad de comunicación multipunto. Estas características lo convierten en una opción atractiva para su incorporación en contextos educativos. Este trabajo describe una red de comunicación basada en arquitectura maestro-esclavo compuesta por módulos ESP32, que permite explorar conceptos como direccionamiento de dispositivos, sincronización y transmisión Half-Duplex.

II. ARQUITECTURA DEL SISTEMA

El sistema está conformado por un nodo maestro y tres nodos esclavos conectados mediante un bus RS-485. En la Figura 1 se puede observar el diagrama de bloques del sistema.

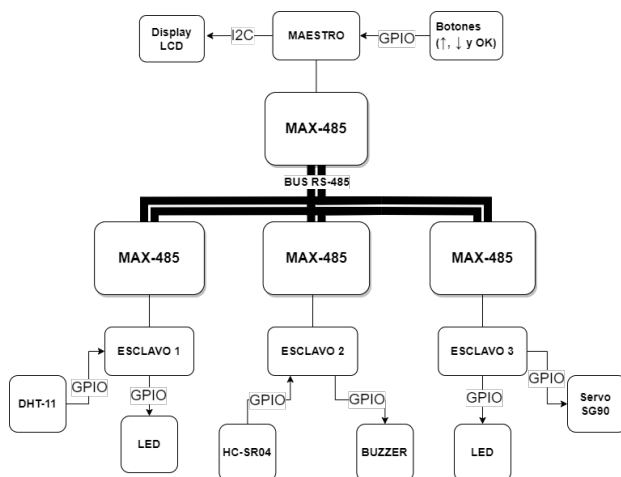


Figura 1. Diagrama en bloques del sistema maestro-esclavo.

A. Maestro

El maestro transmite tramas de 8 bits con la siguiente estructura:

Tabla I
ESTRUCTURA DE LA TRAMA ENVIADA POR EL MAESTRO.

7	6	5	4	3	2	1	0
CRC			Función			ID	

Donde:

- **ID (2 bits)**: Identifica el esclavo al que está dirigida la trama.
- **Función (3 bits)**: Código de la acción solicitada (encender LED, leer sensor, etc.).
- **CRC (3 bits)**: Campo de verificación, constante en esta implementación.

El usuario puede seleccionar funciones mediante tres botones físicos y visualizar la información en una pantalla LCD I2C de 16x2.

B. Esclavos

Cada esclavo posee una dirección única y está configurado para ejecutar funciones específicas, detalladas mas adelante.

III. DISEÑO DEL SISTEMA

A. Comunicación

Se utilizan módulos MAX485 para la conversión de niveles TTL a RS-485. El bus opera en modo Half-Duplex, alternando entre transmisión y recepción sobre un único par de cables.

B. Protocolo

El estándar RS-485, de acuerdo a la especificación EIA/TIA-485, permite conectar hasta 32 cargas unitarias en un mismo bus, con alcances de hasta 1200 metros y velocidades de hasta 10 Mbps en distancias cortas. No obstante, en nuestra implementación se utiliza una adaptación del protocolo RS-485 que simplifica y optimiza la comunicación para las necesidades del sistema.

En nuestra versión adaptada se ha diseñado un protocolo de comunicación en el cual los mensajes se estructuran de forma compacta en un único byte, dividiendo dicho byte en tres campos: (i) identificación del dispositivo, (ii) código de función, y (iii) un valor de verificación (CRC) constante. Esta adaptación facilita la gestión de la comunicación en sistemas embebidos con recursos limitados.

C. Resistencias de Terminación

Una red RS-485 bien terminada requiere resistencias cuya impedancia coincida con la del cableado utilizado, lo que evita reflexiones y distorsiones en la señal. Sin embargo, para redes cortas y de baja velocidad —como este caso—, es común prescindir de estas resistencias para simplificar la implementación.

IV. HARDWARE E IMPLEMENTACIÓN

El sistema completo se observa en la Figura 2, fue implementado a partir de una arquitectura modular compuesta por un nodo maestro y tres nodos esclavos, todos interconectados mediante un único bus físico RS-485. Cada nodo se basa en un microcontrolador ESP32, el cual se comunica a través de un transceptor MAX485 que convierte los niveles lógicos TTL a los niveles diferenciales requeridos por el estándar RS-485.

Todos los nodos fueron alimentados mediante una fuente regulada de 5 V conectada al pin V_{in} de cada ESP32. La interconexión entre los módulos se realizó utilizando cableado con distancias cortas que no superan los 5 metros, lo cual permitió prescindir del uso de resistencias de terminación.

A continuación, se describe en detalle el hardware correspondiente al nodo maestro y a los nodos esclavos.

A. Maestro

El maestro fue diseñado con una interfaz de usuario basada en tres botones físicos que permiten navegar por un menú visible en un display LCD. La elección de esta interfaz busca simular un entorno de control real, donde el usuario selecciona dispositivo esclavo y funciones de forma directa.

El hardware del maestro incluye un ESP32, un transceptor MAX485, tres botones (GPIO 12, 13, 14) y un display LCD I2C, esta interfaz permite reducir la cantidad de pines requeridos, facilitando el cableado y la programación..

B. Esclavos

Cada esclavo fue construido utilizando un módulo ESP32 y un transceptor MAX485 para su conexión al bus RS-485. A continuación, se describen los dispositivos conectados en cada esclavo.

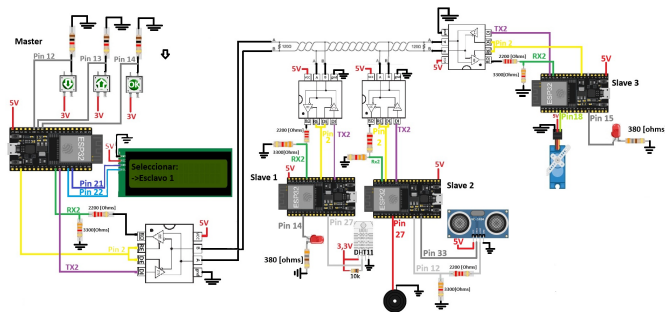


Figura 2. Diagrama de conexiones del sistema.

C. Hardware de los Esclavos

Cada esclavo fue diseñado para controlar o leer dispositivos periféricos específicos. La distribución de sensores y actuadores por nodo se detalla a continuación:

- Esclavo 1
 - LED
 - Sensor DHT11
- Esclavo 2
 - Sensor HC-SR04
 - Buzzer
- Esclavo 3
 - Servo motor SG90
 - LED

D. Firmware

El firmware del sistema fue desarrollado utilizando el entorno de programación de Arduino, tanto para el nodo maestro como para los esclavos. Se optó por una arquitectura basada en máquinas de estados finitos (MEF), debido a su simplicidad y capacidad para organizar de forma clara los flujos de control y eventos del sistema. En las figuras 3 y 4 se observan las MEFs correspondientes al maestro y a los esclavos respectivamente.

En el caso del maestro, la MEF está compuesta por varios estados definidos, que permiten la navegación por el menú, la selección del esclavo y la función deseada, la generación de la trama correspondiente y la espera de respuesta. Una vez seleccionada la función, el maestro codifica la información en una trama de 8 bits y la transmite por el bus RS-485. A continuación, espera la respuesta del nodo esclavo correspondiente y actualiza el display con la información obtenida. En el caso que el maestro no reciba una respuesta del esclavo dentro de un tiempo establecido, se activa una rutina de manejo de errores. Esta rutina informa que el esclavo no está conectado mediante un mensaje en el display LCD y luego retorna al menú principal, permitiendo al usuario realizar una nueva selección o intentar la comunicación nuevamente.

En los nodos esclavos, el firmware también se estructura en forma de máquina de estados finitos (MEF). Inicialmente, cada esclavo se encuentra en un estado de escucha pasiva sobre el bus RS-485. Cuando se detecta una trama entrante, se

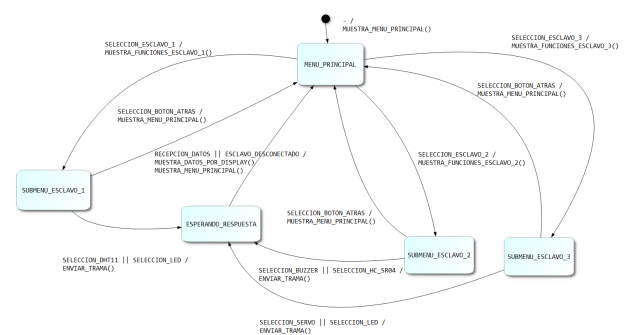


Figura 3. Máquina de estados finita del maestro.

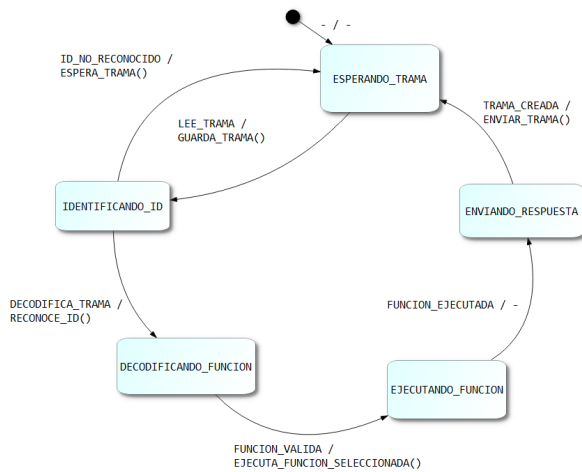


Figura 4. Máquina de estados de los esclavos.

verifica si la dirección contenida en los bits correspondientes coincide con la ID asignada al dispositivo. En caso afirmativo, se decodifica el código de función y se ejecuta la acción correspondiente, que puede implicar la activación de un actuador o la toma de una medición. Luego de ejecutar la función, el esclavo genera una trama de respuesta que es enviada al maestro, sin importar el tipo de acción solicitada. Esta confirmación asegura que el maestro pueda verificar la ejecución de la orden, incluso en comandos sin retorno de datos, como por ejemplo el encendido de un LED o la activación del buzzer.

V. DESAFÍOS Y FUTURAS MEJORAS

Durante la implementación y las pruebas se identificaron diversos desafíos técnicos que abren oportunidades de mejora para futuras versiones del sistema. Entre los principales se destacan:

- **Gestión de interferencias y terminación del bus:** Si bien en el presente prototipo se prescindió del uso de resistencias de terminación debido a la corta longitud del cableado, en aplicaciones a mayor escala resulta indispensable incorporar resistencias de aproximadamente 120 Ω en los extremos del bus RS-485. Esta medida permite absorber correctamente las señales reflejadas y minimizar errores de comunicación.
- **Ampliación del protocolo de comunicación:** La versión actual utiliza tramas de 8 bits con campos fijos para identificación, función y verificación simple. La incorporación de un algoritmo de CRC dinámico permitiría una detección más robusta de errores en ambientes con mayor interferencia. Asimismo, el aumento del tamaño de las tramas habilitaría la transmisión de datos más complejos o múltiples parámetros en un solo mensaje.
- **Optimización del firmware mediante interrupciones:** La lógica actual se basa en una máquina de estados finitos (MEF) secuencial. Se propone migrar hacia una arquitectura basada en interrupciones, que permitiría atender eventos de manera inmediata y no bloqueante.

Esta mejora facilitaría la ejecución concurrente de tareas y optimizaría el consumo energético mediante modos de bajo consumo entre eventos.

VI. RESULTADOS

El sistema propuesto fue sometido a un conjunto de pruebas funcionales con el objetivo de validar su desempeño en condiciones reales de laboratorio. Durante estas pruebas se evaluó la comunicación entre el nodo maestro y cada uno de los nodos esclavos, así como la respuesta de los dispositivos conectados ante las funciones enviadas.

Los resultados obtenidos fueron positivos en todos los casos, destacándose la estabilidad de la comunicación RS-485, la correcta interpretación de las tramas por parte de los nodos esclavos y la ejecución precisa de las acciones requeridas. A continuación, se describen los aspectos verificados durante las pruebas:

- **Respuesta a comandos del maestro:** Cada esclavo respondió correctamente a los comandos recibidos, ejecutando las acciones solicitadas sin errores de comunicación o lógica.
- **Lecturas estables de sensores:** En las pruebas realizadas con los sensores DHT11 y HC-SR04, se observaron lecturas consistentes, dentro de los márgenes esperados. No se detectaron fluctuaciones erráticas ni errores de lectura.
- **Activación confiable de actuadores:** Los actuadores conectados (LEDs, buzzer y servo motor) reaccionaron de forma inmediata y precisa a las órdenes enviadas por el maestro, sin demoras perceptibles ni fallos de ejecución.
- **Detección de desconexión de esclavos:** El sistema fue capaz de identificar la ausencia de respuesta cuando un esclavo estaba desconectado, lo cual permitió confirmar la funcionalidad del manejo de excepciones en la lógica del maestro.

Estas validaciones se complementaron con observaciones visuales a través de la interfaz LCD del maestro y mediante el monitoreo en consola serie. En todos los casos, el comportamiento del sistema se mantuvo dentro de los parámetros esperados.

A continuación, se presentan evidencias gráficas que ilustran algunos de los escenarios de prueba realizados sobre el sistema maestro-esclavo.

VII. CONCLUSIONES

El sistema desarrollado demuestra una implementación eficiente y funcional de una red de comunicación RS-485 utilizando microcontroladores ESP-32. Gracias a su arquitectura distribuida, el nodo maestro puede controlar múltiples dispositivos esclavos, cada uno con funciones específicas, como la lectura de sensores y el control de actuadores. La integración de una pantalla LCD y botones de navegación facilita la interacción del usuario, permitiendo seleccionar nodos y visualizar información de manera clara.

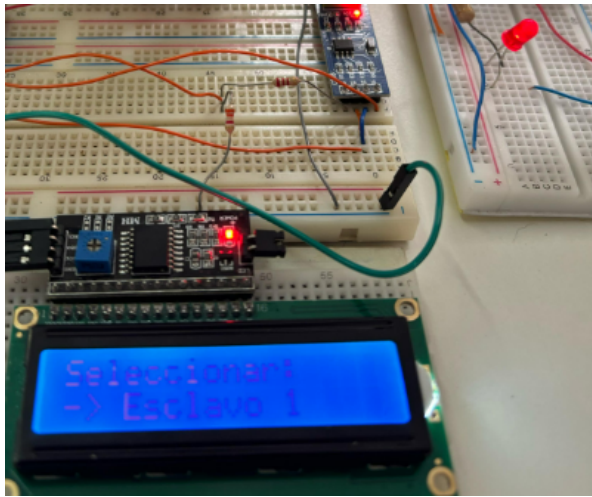


Figura 5. Prueba de encendido de LED (Esclavo 1).

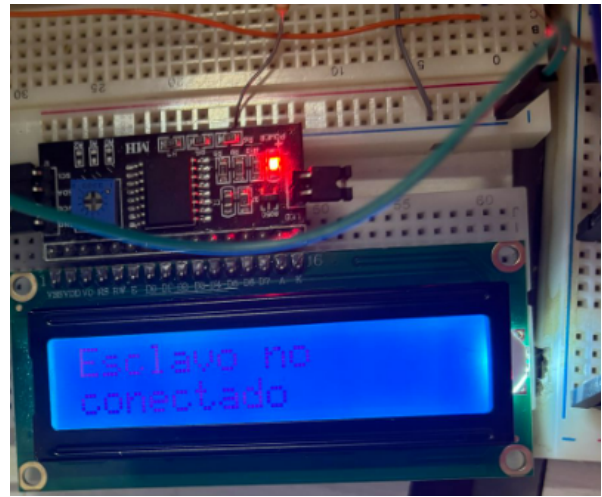


Figura 8. Detección de esclavo desconectado.

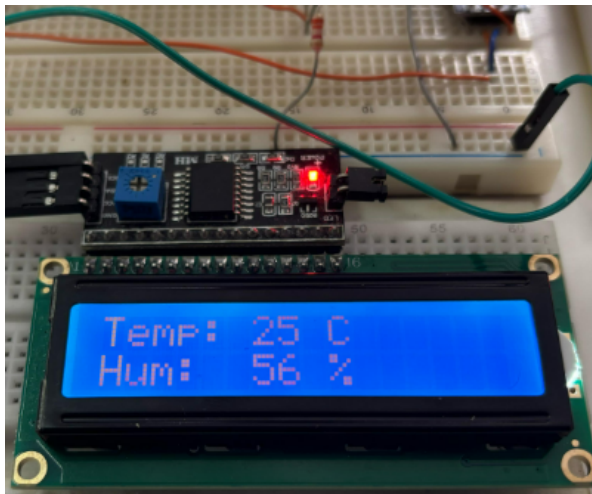


Figura 6. Lectura de temperatura y humedad (Esclavo 1).



Figura 7. Lectura del sensor ultrasónico HC-SR04 (Esclavo 2).

REFERENCIAS

- [1] Espressif Systems, *ESP32 Technical Reference Manual*, versión 4.1, Espressif Systems, 2020.
- [2] Texas Instruments, *RS-485 Basics Series*, Application Report SLLA070D, Texas Instruments Inc., 2016.
- [3] NOVUS, *Conceptos Fundamentales de RS485 y RS422*, NOVUS Productos Eletrônicos Ltda., 2015.
- [4] Electronic Team Inc., *Guía de la comunicación RS485*, Electronic Team, 2017.
- [5] Aosong Electronics, *DHT11 Humidity and Temperature Sensor Datasheet*, versión 1.3, Aosong Co. Ltd., 2018.
- [6] Tower Pro, *SG90 9G Micro Servo Datasheet*, Tower Pro, 2014.
- [7] Dejan Nedelkovski, *PWM Control of SG90 Servo with ESP32*, HowTo-Mechatronics, 2019.
- [8] Maxim Integrated, *MAX485—Low-Power, Slew-Rate-Limited RS-485/RS-422 Transceivers*, Datasheet No. 19-1284; Rev 12, Maxim Integrated, 2016.
- [9] Departamento de Electrónica, *Apuntes de la asignatura Interfaces*, Universidad Nacional de San Luis, 2024.